

Unima - Afya
Curso de Ciência da Computação

IMPLEMENTAÇÃO DE UM OTIMIZADOR DE BLOCOS BÁSICOS

Otimização Local de Instruções Lineares via Constant Folding, Constant
Propagation e Dead Code Elimination

Bruno Santos Moraes
Fabio Fernandes Reis Filho
João Honório Barbosa

Maceió – AL

2026

Sumário

1	Introdução	3
2	Metodologia de Implementação	4
2.1	Parser e Representação Intermediária (IR)	4
2.2	Constant Folding e Propagation	4
2.3	Dead Code Elimination	5
2.4	Laço de Otimização em Ponto Fixo	5
3	Casos de Teste e Discussão	5
3.1	Caso de Teste 1: Constant Folding e Propagation Básico	5
3.2	Caso de Teste 2: Eliminação de Código Morto Simples	6
3.3	Caso de Teste 3: Ponto Fixo Complexo	7
4	Conclusão	7
5	Referências	8

1. Introdução

No ciclo de desenvolvimento de um compilador, a fase de otimização de código intermediário desempenha um papel crítico para melhorar o desempenho e reduzir a pegada de processamento e memória do programa final. O objetivo primário de um otimizador é aplicar transformações semânticas seguras que resultem em código executado de forma mais rápida ou ocupando menos espaço de armazenamento.

As otimizações de código dividem-se hierarquicamente pelo seu escopo de atuação. O nível mais elementar e fundamental é a otimização local, que é aplicada sobre **Blocos Básicos**. Um bloco básico é caracterizado como uma sequência de instruções lineares (código de três endereços) que possui um único ponto de entrada (a primeira instrução) e um único ponto de saída (a última instrução), não contendo quaisquer instruções de desvios (como jumps condicionados ou não) internos.

Este trabalho apresenta a fundamentação teórica, a metodologia de projeto e os resultados de validação de um otimizador de blocos básicos implementado em Python. O sistema foca no desenvolvimento prático de três técnicas clássicas da teoria de compiladores:

- **Constant Folding (Dobramento de Constantes):** Técnica que avalia expressões matemáticas, relacionais ou lógicas cujos operandos sejam constantes estáticas conhecidas em tempo de compilação, substituindo a operação complexa por seu valor resultante literal direto.
- **Constant Propagation (Propagação de Constantes):** Técnica executada em conjunto com o dobramento de constantes, que consiste em substituir referências futuras de variáveis locais pelos seus respectivos valores literais constantes caso esses valores sejam previsíveis de forma estática no fluxo de execução do bloco básico.
- **Dead Code Elimination - DCE (Eliminação de Código Morto):** Identifica e remove instruções locais que realizam atribuições em variáveis temporárias ou locais que não são lidas em nenhuma instrução subsequente do bloco básico (variáveis mortas), de forma a reduzir a contagem de instruções redundantes.

2. Metodologia de Implementação

O otimizador foi desenvolvido em Python de forma modular. A estrutura do código-fonte é composta por quatro etapas principais descritas abaixo.

2.1. Parser e Representação Intermediária (IR)

A entrada aceita pelo otimizador é uma sequência linear de strings contendo Código de Três Endereços (TAC). Cada instrução é convertida em um objeto da classe `Instruction`, modelando a instrução como uma tupla do tipo:

$$\text{Destino}(\text{target}) \leftarrow \text{Operando1}(\text{arg1}) \quad \text{Operador}(\text{op}) \quad \text{Operando2}(\text{arg2})$$

O parser identifica atribuições simples, operações binárias (aritméticas, booleanas, relacionais), operações unárias e comandos finais de retorno de fluxo (`return`).

2.2. Constant Folding e Propagation

O fluxo de análise é descendente (do início do bloco para o fim). É mantido um dicionário de símbolos constantes denotado por `constants`. Para cada instrução do bloco:

1. **Substituição:** Se `arg1` ou `arg2` forem identificadores contidos na tabela `constants`, eles são substituídos por seus valores literais direto.
2. **Avaliação (Folding):** Se a instrução possuir um operador e ambos os argumentos forem constantes (ou o argumento único de uma instrução unária), a operação é avaliada no compilador e a instrução original é substituída por uma atribuição direta de constante.
3. **Atualização de Contexto:** Se a instrução final for do tipo `target = constante`, o dicionário de constantes é atualizado com o valor atribuído a `target`. Caso o `target` receba uma variável dinâmica ou valor não constante, a entrada correspondente em `constants` é imediatamente invalidada e removida.

2.3. Dead Code Elimination

A eliminação de código morto opera em sentido reverso (ascendente, do fim do bloco para o início). Inicialmente, é definido o conjunto de variáveis ativas de saída do bloco (`live_vars`). Se a instrução de saída for do tipo `return x`, a variável `x` inicia ativa. Para cada instrução, subindo no bloco básico:

1. Se a instrução define uma variável (`target`) e esta **não** está contida no conjunto `live_vars`, a linha inteira é classificada como código morto e é removida do bloco físico de instruções.
2. Se a variável `target` estiver contida em `live_vars`, a instrução é mantida viva. A definição de `target` mata sua liveness anterior (removendo-a de `live_vars`), e os argumentos lidos na instrução (`arg1` e `arg2`) são adicionados a `live_vars`, pois precisam estar vivos para alimentar esta linha.

2.4. Laço de Otimização em Ponto Fixo

Como a simplificação por Constant Folding pode tornar variáveis redundantes (código morto) e a remoção de código morto pode eliminar redefinições permitindo novas propagações, o compilador encapsula ambos os passes em um laço de repetição. A otimização executa seguidas vezes até que uma iteração completa não produza nenhuma modificação nas instruções, caracterizando a convergência no **ponto fixo** ótimo.

3. Casos de Teste e Discussão

A corretude e eficácia do otimizador foram demonstradas em três cenários práticos e validadas por meio de testes unitários automatizados.

3.1. Caso de Teste 1: Constant Folding e Propagation Básico

Este caso de teste demonstra o funcionamento das propagações de constantes em cascata em um único bloco.

```
a = 2
b = 3
```

```
c = a + b
d = c * 4
return d
```

Listing 1: Código original de entrada (Caso 1)

O processo de otimização em ponto fixo ocorre em duas iterações:

- No passo de Constant Folding, as constantes `a = 2` e `b = 3` substituem a expressão em `c`, que vira `2 + 3` e é dobrada para 5. O valor de `c` é propagado em `d`, virando `5 * 4` que dobra para 20. O retorno é alterado para `return 20`.
- No passo de Dead Code Elimination, as definições locais de `a`, `b`, `c` e `d` são avaliadas como mortas, pois a saída agora é uma constante literal direta.

O código final otimizado resulta em:

```
return 20
```

Listing 2: Código otimizado (Caso 1)

Esta transformação representa uma redução de **80%** na contagem física de instruções no bloco.

3.2. Caso de Teste 2: Eliminação de Código Morto Simples

Este cenário demonstra a habilidade do otimizador de remover variáveis não utilizadas que não influenciam no resultado de saída.

```
x = 10
y = 20
z = x + 5
unused_var = 500
return z
```

Listing 3: Código original de entrada (Caso 2)

A análise ascendente de liveness determina que a variável final viva na saída é apenas `z`. O algoritmo descarta a instrução `unused_var = 500` e `y = 20` por não estarem em `live_vars`. Se o ponto fixo for executado na sua integridade, o compilador propaga `x = 10` em `z`, dobra `10 + 5` para 15 e remove também `x` e `z`, gerando:

```
return 15
```

Listing 4: Código otimizado em ponto fixo (Caso 2)

3.3. Caso de Teste 3: Ponto Fixo Complexo

Este caso de teste une a dependência múltipla de constantes intermediárias com cálculo de código morto.

```
a = 10
b = 20
c = a + b
d = c * 2
e = d + 100
return c
```

Listing 5: Código original de entrada (Caso 3)

- **Fase de Dobramento:** O motor avalia `c` como 30, `d` como 60 e `e` como 160. A instrução de saída é atualizada para `return 30`.
- **Fase de DCE:** Como o retorno é a constante literal 30, nenhuma variável do bloco é considerada viva na saída. Todas as instruções que calculam as variáveis locais e temporárias (`a`, `b`, `c`, `d`, `e`) são removidas.

O código final reduzido é de apenas:

```
return 30
```

Listing 6: Código otimizado (Caso 3)

4. Conclusão

A aplicação prática das técnicas de Constant Folding, Constant Propagation e Dead Code Elimination sobre blocos básicos lineares demonstrou ser altamente eficaz. Ao reduzir os cálculos que seriam efetuados dinamicamente em tempo de execução para avaliações estáticas na compilação, eliminamos gargalos de instruções na CPU.

A implementação baseada em ponto fixo é segura e atinge a convergência de maneira otimizada em todos os cenários. Em ambientes de compiladores acadêmicos ou comerciais reais, a aplicação de otimizações de bloco básico garante menor sobrecarga nas fases posteriores de seleção de instruções e alocação de registradores físicos no processador.

5. Referências

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Addison-Wesley, 2008.

MUCHNICK, Steven S. *Advanced compiler design and implementation*. San Francisco: Morgan Kaufmann, 1997.

COOPER, Keith D.; TORCZON, Linda. *Engineering a compiler*. 2. ed. San Francisco: Morgan Kaufmann, 2011.